

TCC70xx MCU BSP

API Specification for Local Interconnect Network

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files.....	4
4	Enumerations	5
4.1	LINMode_t.....	5
4.2	LinStatus_t.....	6
5	Structures	7
5.1	LINData_t.....	7
5.2	LINConfig_t.....	8
6	Callback Functions	9
6.1	LINResponseIsrFunc	9
7	APIs	10
7.1	LIN_Init.....	10
7.2	LIN_MasterTx.....	11
7.3	LIN_MasterRx	12
7.4	LIN_ReadHeader	13
7.5	LIN_ReadResponse.....	14
7.6	LIN_CalcProtectId.....	15
7.7	LIN_CalcFrameId.....	16
7.8	LIN_CalClassicChecksum	17
7.9	LIN_CalEnhancedChecksum.....	18
7.10	LIN_FlushRxData.....	19
7.11	LIN_RegisterCB	20
8	How to use LIN Driver.....	21
8.1	Initialize LIN Driver	21
8.2	Send Header and Response as Master Mode	22
8.3	Send Header and Read Response as Master Mode	23
8.4	Read Header and Response as Slave Mode	23
8.5	Read Header and Send Response as Slave Mode	24
9	References	25
10	Revision History	26
	Rev. 0.10: 2021-09-03	26

Figures

Figure 3.1 Location of Source Files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the local interconnect network device driver (hereinafter referred to as LIN driver) of TCC70xx MCU BSP. The LIN is a serial communications protocol which efficiently supports the control of mechatronics nodes in distributed automotive applications. For more detailed information on Local Interconnect Network, refer to "*ISO standard ISO 17987 Part 1-7*".

Telechips does not have LIN Bus hardware, but you can operate LIN communication using Universal Asynchronous Receiver/Transmitter (UART) interface hardware and external LIN transceiver. For more detailed information on UART interface hardware in the TCC70xx, refer to Chapter 10 Universal Asynchronous Receiver/Transmitter (UART) in "*TCC70xx Full Specification*".

[1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

LIN	Local Interconnect Network
PID	Protected Identifier
UART	Universal Asynchronous Receiver/Transmitter

3 SOURCE FILES

3.1 Location of Source Files

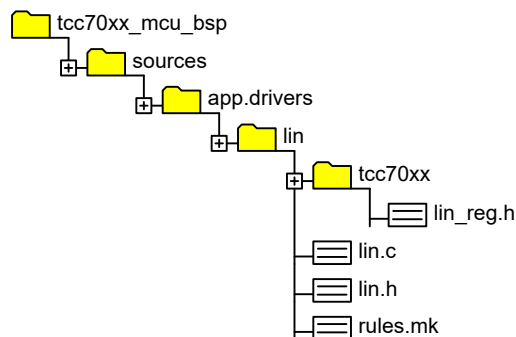


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `lin_reg.h`
 - Define variables that are dependent on hardware
- `lin.c`
 - LIN driver source
- `lin.h`
 - LIN driver API header
- `rules.mk`
 - Rules used for build

4 ENUMERATIONS

4.1 LINMode_t

Enumeration variables for LIN Mode

Declaration

```
typedef enum LINMode
{
    LIN_MASTER    = 0,
    LIN_SLAVE
} LINMode_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>LIN_MASTER</i>	<i>Send Header and Response, Read Response</i>
<i>LIN_SLAVE</i>	<i>Read Header and Response</i>

Remark

None

4.2 LinStatus_t

Enumeration variables for current LIN status

Declaration

```
typedef enum LINStatus
{
    LIN_ERROR           = -1,
    LIN_OK              = 0,
    LIN_HEADER_ERROR,
    LIN_NO_RESPONSE,
    LIN_CSUM_ERROR,
    LIN_NO_DATA_INBUS,
} LinStatus_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>LIN_ERROR</i>	<i>LIN error occurred</i>
<i>LIN_OK</i>	<i>No error</i>
<i>LIN_HEADER_ERROR</i>	<i>Clarify LIN Header Break Field, Sync Field</i>
<i>LIN_NO_RESPONSE</i>	<i>Transmitting Header is success.</i>
<i>LIN_CSUM_ERROR</i>	<i>Calculated Checksum and Received Checksum are <u>not</u> the same.</i>
<i>LIN_NO_DATA_INBUS</i>	<i>Fail to read any data from LIN bus</i>

Remark

None

5 STRUCTURES

5.1 LINData_t

This struct includes LIN Communication Protocol Information.

Declaration

```
typedef struct LINData
{
    uint8 LdPid;
    uint8 LdData[LIN_MAX_DATA_SIZE];
    uint8 LdCSum;
    uint8 LdSize;
    uint8 LdDLC;
} LINData_t;
```

Description

Value	Description
LdPid	Protected ID of LIN communication
LdData	Data that will transmit/receive through LIN bus
LdCSum	Checksum value that will transmit/receive through LIN bus
LdSize	Send/Read Data Size. <u>Not</u> including header and checksum size
LdDLC	Data Length Size fixed by Ldf file

Remark

None

5.2 LINConfig_t

This struct includes configuration value for LIN driver.

Declaration

```
typedef struct LINConfig
{
    uint32    cfgLinCh;
    uint32    cfgPortSel;
    uint32    cfgBaud;
    uint32    cfgLinSlpPin;
    uint8     cfgMode;           //Polling, Interrupt
    uint8     cfgGicPriority;
    LINMode_t cfgLinMode;
} LINConfig_t;
```

Description

Value	Description
<i>cfgLinCh</i>	<i>UART Controller number (0 to 5)</i>
<i>cfgPortSel</i>	<i>Channel to Port Mapping Information of UART. This value is saved in Port Selection Register.</i>
<i>cfgBaud</i>	<i>Baud Rate of LIN communication</i>
<i>cfgLinSlpPin</i>	<i>GPIO Port used as LIN Transceiver Sleep Control Pin</i>
<i>cfgMode</i>	<i>Polling/Interrupt/DMA Mode</i>
<i>cfgGicPriority</i>	<i>Interrupt Priority</i>
<i>cfgLinMode</i>	<i>LIN Master/Slave Mode</i>

Remark

None

6 CALLBACK FUNCTIONS

6.1 LINResponseIsrFunc

Callback function for sending response frame.

Declaration

```
typedef void (*LINResponseIsrFunc)
(
    void * pArg,
    uint8 ucPid
);
```

Parameters

Value	Description
pArg	[in] UartStatus_t struct
ucPid	[in] Protected identifier

Return

None

Remark

None

7 APIs

7.1 LIN_Init

Initializes the LIN driver, sets the baud rate, UART channel, UART port selection, and wakes up the LIN transceiver.

Declaration

```
sint8 LIN_Init
(
    LINConfig_t    sLinCfg
);
```

Parameters

Value	Description
sLinCfg	[in] The specific configuration. Refer to Chapter 5.2 LINConfig_t.

Return

Value	Description
LIN_OK	Success
others	Failure

Remark

None

7.2 LIN_MasterTx

Sends LIN Header Frame and Response Frame in Master Mode.

Declaration

```
sint8 LIN_MasterTx  
(  
    LINConfig_t    sLinCfg,  
    LINData_t *    psLinTxData  
);
```

Parameters

Value	Description
<i>sLinCfg</i>	<i>[in] LIN Driver Config</i>
<i>psLinTxData</i>	<i>[in] Include LIN Communication Protect ID, Data, Checksum, Data Size</i>

Return

Value	Description
<i>LIN_OK</i>	<i>Success</i>
<i>others</i>	<i>Failure</i>

Remark

None

7.3 LIN_MasterRx

Sends only LIN Header Frame in Master Mode.

Declaration

```
sint8 LIN_MasterRx
(
    LINConfig_t    sLinCfg,
    LINData_t *    pSLinTxData
);
```

Parameters

Value	Description
sLinCfg	[in] LIN Driver Config
pSLinTxData	[in] Send LIN Header Frame Only

Return

Value	Description
LIN_OK	Success
others	Failure

Remark

None

7.4 LIN_ReadHeader

Reads and clarifies LIN Header Frame.

Declaration

```
sint8 LIN_ReadHeader  
(  
    LINConfig_t    sLinCfg,  
    uint8 *        pucPid  
);
```

Parameters

Value	Description
<i>sLinCfg</i>	<i>[in] LIN Driver Config</i>
<i>pucPid</i>	<i>[out] Get LIN Protected identifier</i>

Return

Value	Description
<i>LIN_ERROR</i>	<i>Undefined Error</i>
<i>LIN_NO_DATA_INBUS</i>	<i>There is <u>no</u> data in LIN bus.</i>
<i>LIN_NO_RESPONSE</i>	<i>Success to read Header Frame. You can read Response Frame.</i>
<i>LIN_HEADER_ERROR</i>	<i>Break field or sync field is wrong.</i>

Remark

None

7.5 LIN_ReadResponse

Reads and clarifies LIN Response Frame.

Declaration

```
sint8 LIN_ReadResponse  
(  
    LINConfig_t    sLinCfg,  
    LINData_t *    psLinRxData  
);
```

Parameters

Value	Description
<i>sLinCfg</i>	<i>[in] LIN Driver Config</i>
<i>psLinRxData</i>	<i>[in] LdPid of LINData_t struct</i> <i>[in] LdDLS of LINData_t struct is set from .ldf file</i> <i>[out] LdData, LdCSum, LdSize of LINData_t struct are read from LIN bus.</i>

Return

Value	Description
<i>LIN_ERROR</i>	<i>Rx Data Buffer is NULL.</i>
<i>LIN_CSUM_ERROR</i>	<i>Calculated Checksum and Received Checksum are <u>not</u> the same.</i>
<i>LIN_NO_DATA_INBUS</i>	<i>There is no data in LIN bus.</i>

Remark

None

7.6 LIN_CalcProtectId

Calculates the Protected identifier from the Frame identifier.

Declaration

```
sint8 LIN_CalcProtectId
(
    uint8      ucFid,
    uint8 *    pucPid
);
```

Parameters

Value	Description
ucFid	[in] Frame identifier
pucPid	[out] Protected identifier

Return

Value	Description
LIN_OK	Success
LIN_ERROR	Failure

Remark

None

7.7 LIN_CalcFrameId

Calculates the Frame identifier from the Protected identifier.

Declaration

```
sint8 LIN_CalcFrameId
(
    uint8      ucPid,
    uint8 *    pucFid
);
```

Parameters

Value	Description
ucPid	[in] Protected identifier
pucFid	[out] Frame identifier

Return

Value	Description
LIN_OK	Success
LIN_ERROR	Failure

Remark

None

7.8 LIN_CalClassicChecksum

Calculating a checksum using only data bytes is called a classic checksum.

Declaration

```
uint8 LIN_CalClassicChecksum
(
    uint8 *    pucData,
    uint8      ucSize
);
```

Parameters

Value	Description
<i>pucData</i>	<i>[in] Data</i>
<i>ucSize</i>	<i>[in] Size of data</i>

Return

Value	Description
<i>others</i>	<i>Classic checksum</i>
<i>LIN_ERROR</i>	<i>Failure</i>

Remark

None

7.9 LIN_CalEnhancedChecksum

The Enhanced checksum model used in the LIN specification versions starting from LIN 2.0 for all frames, except the diagnostic frames.

Declaration

```
uint8 LIN_CalEnhancedChecksum
(
    uint8      ucPid,
    uint8 *    pucData,
    uint8      ucSize
);
```

Parameters

Value	Description
ucPid	[in] Protected identifier
pucData	[in] Data
ucSize	[in] Size of data

Return

Value	Description
others	Enhanced checksum
LIN_ERROR	Failure

Remark

None

7.10 LIN_FlushRxData

Flushes Rx data of LIN.

Declaration

```
sint8 LIN_FlushRxData  
(  
    LINConfig_t    sLinCfg  
);
```

Parameters

<i>Value</i>	<i>Description</i>
<i>sLinCfg</i>	<i>[in]</i> LIN Driver Config

Return

<i>Value</i>	<i>Description</i>
<i>cRxsize</i>	<i>Flush data size</i>
<i>LIN_ERROR</i>	<i>Failure</i>

Remark

None

7.11 LIN_RegisterCB

Registers callback function for sending LIN Response frame.

Declaration

```
void LIN_RegisterCB
(
    LINResponseIsrFunc    pLinResFunc
);
```

Parameters

Value	Description
pLinResFunc	[in] Pointer of LIN response operation callback function

Return

None

Remark

None

8 HOW TO USE LIN DRIVER

8.1 Initialize LIN Driver

```
{  
    ...  
  
    LINConfig_t sLinConfig;  
  
    sLinConfig.uiLinCh          = LINTEST_CH ;  
    sLinConfig.uiBaud          = LINTEST_BAUDRATE ;  
    sLinConfig.uiPortSel       = LINTEST_PORTSEL ;  
    sLinConfig.uiLinSlpPin     = LINTEST_SLP ;  
    sLinConfig.cfgMode         = UART_DMA_MODE;  
    sLinConfig.cfgLinMode      = LIN_MASTER;  
    sLinConfig.cfgGicPriority   = LIN_GIC_PRIORITY;  
  
    LIN_Init(sLinConfig);  
  
    ...  
}
```

8.2 Send Header and Response as Master Mode

```
sLinConfig.cfgLinMode    = LIN_MASTER;

{
    ...

    uint8 uiSendId        = 0x11;
    uint8 ucSendData[3]   = {0xAA, 0xBB, 0xCC};
    uint8 ucSendSize      = 3;

    LIN_SendData(sLinConfig, uiSendId, ucSendData, ucSendSize);

    var_ID                = (uint8)MSEND_TEST_FID;
    sLINData.LdSize       = 3;

    SAL_MemCopy(sLINData.LdData, ucSendData, sLINData.LdSize);

    /* Calculate Protected identifier */
    LIN_CalcProtectId(var_ID, &sLINData.LdPid);

    /* Calculate Checksum */
    if ((var_ID == LIN_DIAGNOSIC_ID1) || (var_ID == LIN_DIAGNOSIC_ID2))
    {
        sLINData.LdCsum = LIN_CalClassicChecksum(sLINData.LdData, sLINData.LdSize);
    }
    else
    {
        sLINData.LdCsum = LIN_CalEnhancedChecksum(sLINData.LdPid, sLINData.LdData, sLINData.LdSize);
    }

    LIN_MasterTx(sLinConfig, &sLINData);

    ...
}
```

8.3 Send Header and Read Response as Master Mode

```
sLinConfig.cfgLinMode    = LIN_MASTER;

{
    ...

    var_ID = (uint8)MREAD_TEST_FID;

    /* Calculate identifier */
    LIN_CalcProtectId(var_ID, &sLINData.LdPid);
    LIN_MasterRx(sLinConfig, &sLINData);

    ...

    cRet = LIN_ReadHeader(sLinConfig, &sLINRxData.LdPid);

    if(cRet == LIN_NO_RESPONSE)
    {
        LIN_CalcFrameId(sLINRxData.LdPid, &ucFid);
        sLINRxData.LdDLC = LIN_GetDLC(ucFid);

        cRet = LIN_ReadResponse(sLinConfig, &sLINRxData);
    }
}
```

8.4 Read Header and Response as Slave Mode

```
sLinConfig.cfgLinMode    = LIN_SLAVE;

{
    ...

    cRet = LIN_ReadHeader(sLinConfig, &sLINRxData.LdPid);

    if(cRet == LIN_NO_RESPONSE)
    {
        LIN_CalcFrameId(sLINRxData.LdPid, &ucFid);
        sLINRxData.LdDLC = LIN_GetDLC(ucFid);

        cRet = LIN_ReadResponse(sLinConfig, &sLINRxData);
    }

    ...
}
```

8.5 Read Header and Send Response as Slave Mode

```
sLinConfig.cfgLinMode      = LIN_SLAVE;

{
    ...
    cRet = LIN_ReadHeader(sLinConfig, &sLINRxData.LdPid);

    if(cRet == LIN_NO_RESPONSE)
    {
        LIN_CalcFrameId(sLINRxData.LdPid, &ucFid);
        sLINRxData.LdDLC = LIN_GetDLC(ucFid);

        cRet = LIN_ReadResponse(sLinConfig, &sLINRxData);
    }
}

void LIN_ISR ( ...
{
    ...
    while((SAL_ReadReg(UartBaseAddr + UART_REG_FR) & UART_FR_RXFE) != UART_FR_RXFE)

    uiData = (SAL_ReadReg(UartBaseAddr + UART_REG_DR) & 0xFFUL );

    buf[uartStat->sRxIntr.iHead] = uiData;
    uartStat->sRxIntr.iHead++;
    if(uartStat->sRxIntr.iHead >= uartStat->sRxIntr.iSize)
    {
        uartStat->sRxIntr.iHead = 0;
    }

    /* Clarify Lin Header*/
    if(Receive_Header_Flag == TRUE)
    {
        gpLinResFunc(uartStat, uiData);
        Receive_Header_Flag = FALSE;
    }
}
```

9 REFERENCES

- [1] TCC70xx Full Specification
- [2] TCC70xx MCU BSP-API Specification for Universal Asynchronous Receiver Transmitter
- [3] ISO standard ISO 17987 Part 1-7
- [4] Contact Telechips for more details: sales@telechips.com

10 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."